

OikoNaos
Test Execution Report
Versione 1.2



Data: 28/01/2025

Progetto: OikoNaos	Versione: 1.2
Documento: Test Plan	Data: 28/01/2025

Coordinatore del progetto:

Nome	Matricola
Luigi Potestà	0512120076

Partecipanti:

Nome	Matricola
Luigi Potestà	0512120076
Maria Antonietta Ruggiero	0512121126
Giulia Buonafine	0512119695
Mario Sinopoli	0512119113

Scritto da:	Giulia Buonafine
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
26/01/2026	1.0	Inserita prima versione relativa ai testing di sistema e integrazione	Luigi Potestà Giulia Buonafine
27/01/2026	1.1	Inserita sezione relativa ai testing di unità	Mario Sinopoli Maria Antonietta Ruggiero
28/01/2026	1.2	Revisione e modifica finale pre-consegna	Luigi Potestà Giulia Buonafine Mario Sinopoli Maria Antonietta Ruggiero

Indice

1.	Introduzione.....	4
2.	Ambiente di Collaudo.....	4
3.	Unit Testing	4
4.	Integration Testing	6
5.	System Testing.....	6
6.	Riepilogo Copertura	9

1. Introduzione

Il presente documento certifica l'attività di collaudo software eseguita sul sistema OikoNaos. L'obiettivo è validare la correttezza funzionale dell'applicazione, la coerenza dei dati persistiti e i controlli di accesso di base previsti dal prototipo, attraverso tre livelli di testing eseguiti in sequenza top-down:

1. **System Testing:** verifica end-to-end dei principali flussi utente tramite automazione dell'interfaccia web (Selenium), includendo l'interazione completa tra View (JSP), Controller e persistenza dati.
2. **Integration Testing:** verifica della logica di business e dell'integrazione tra Controller (Servlet) e layer di persistenza (DAO), mediante test automatici con JUnit e Mockito, focalizzati su redirect/forward, invocazioni ai DAO e gestione delle condizioni di errore.
3. **Unit Testing:** verifica isolata dei componenti di accesso ai dati (DAO) e delle relative operazioni, con l'obiettivo di validare la correttezza delle query e la coerenza delle informazioni gestite a livello di persistenza.

2. Ambiente di Collaudo

Le attività di collaudo del sistema OikoNaos sono state eseguite in un ambiente di test controllato e coerente con l'architettura prevista per il prototipo, come descritto nel System Design Document.

- **IDE:** IntelliJ IDEA 2024.x
- **Linguaggio di programmazione:** Java 17 (OpenJDK)
- **Framework di Unit e Integration Testing:** JUnit 5 (Jupiter), Mockito 5.x
- **Framework di System Testing:** Selenium WebDriver con browser Google Chrome, gestione automatica dei driver tramite Bonigarcia WebDriverManager
- **Database:**
 - Unit e Integration Testing: accesso al database isolato tramite mocking dei componenti di persistenza (DAO)
 - System Testing: database relazionale MySQL 8.0 reale, utilizzato per la verifica end-to-end dei flussi applicativi

L'application server utilizzato per l'esecuzione dei test di sistema è **Apache Tomcat**, configurato in ambiente locale, al fine di riprodurre fedelmente le condizioni di esecuzione del prototipo

3. Unit Testing

L'obiettivo dei test di unità è verificare in modo isolato la correttezza della logica di accesso ai dati e la gestione dei casi limite (vincoli, eccezioni, condizioni di errore), evitando qualsiasi dipendenza dal database reale o dall'infrastruttura applicativa.

Strategia adottata

Per consentire il testing isolato dei componenti di persistenza senza modificare la struttura dei DAO originali, è stato adottato un approccio basato sul mocking delle dipendenze statiche.

La classe di utilità database.java è stata ottimizzata per supportare il lazy loading della connessione, rendendo possibile l'utilizzo di Mockito (mockStatic) per intercettare le chiamate di accesso al database. Questo approccio ha consentito di simulare Connection, PreparedStatement e ResultSet, permettendo la verifica della logica SQL, della gestione delle eccezioni e dei casi limite in completo isolamento dal database reale, senza introdurre overloading dei metodi né dependency injection esplicita nei DAO.

Risultati per componente

Sottosistema	Classe DAO	Aspetti verificati	Esito
Gestione Spese	TassaDAO, PagamentoDAO, RicevutaDAO	Creazione delle tasse, avvio del flusso di pagamento, generazione delle ricevute e isolamento dal database reale tramite mocking	Superato
Autenticazione e Sicurezza	CredenzialiDAO	Recupero hash password, aggiornamento credenziali, propagazione controllata delle eccezioni	Superato
	TokenResetPasswordDAO	Generazione token sicuri (Base64), persistenza, verifica della scadenza temporale	Superato
Registrazione Utente	CodiceIdentificativoDAO	Validazione del Codice Identificativo Speciale, gestione concorrenza tramite simulazione SELECT ... FOR UPDATE	Superato
Prenotazioni Spazi Comuni	PrenotazioneDAO	Verifica conflitti temporali, prevenzione overbooking, cancellazione sicura delle prenotazioni	Superato
Risorse Condivise	RisorsaDAO	Persistenza risorse, gestione campi opzionali (penali nulle)	Superato
	RichiestaRisorsaDAO	Rilevamento conflitti temporali (esisteConflitto), gestione stati richiesta	Superato
Eventi	EventoDAO	Persistenza eventi, iscrizione e disiscrizione utenti, prevenzione iscrizioni duplicate, rispetto limite partecipanti	Superato
Ticket di Manutenzione	TicketDAO	Inserimento ticket, costruzione dinamica delle query di ricerca (filtri opzionali)	Superato
	AllegatoDAO	Associazione corretta dei metadati degli allegati ai ticket	Superato

La copertura dei test di unità è **intenzionalmente parziale**. Sono state testate in isolamento le componenti critiche di accesso ai dati e le logiche applicative più sensibili, mentre funzionalità secondarie o di puro supporto sono verificate indirettamente tramite test di integrazione e di sistema. Questo approccio è coerente con la strategia di testing adottata per il prototipo.

4. Integration Testing

Verificare che i controller (Servlet) gestiscano correttamente le richieste HTTP, il ciclo di vita della sessione utente e l'orchestrazione dei componenti di persistenza (DAO), assicurando il rispetto delle regole di business definite nel RAD.

Strategia adottata

I test di integrazione sono stati implementati utilizzando JUnit 5 e Mockito, simulando l'ambiente servlet container. In particolare: `HttpServletRequest`, `HttpServletResponse` e `HttpSession` sono stati mockati per controllare input, sessione e redirection; è stato utilizzato `MockedConstruction` di Mockito per intercettare l'istanziamento dei DAO all'interno dei controller, verificando che i metodi corretti venissero invocati; non è stato utilizzato il database reale, in modo da isolare la logica di integrazione Controller-DAO.

Risultati dei Test di Integrazione

ID Test	Componente	Scenario verificato	Esito
IT-AUTH	AutenticazioneController	Login con credenziali valide (hash BCrypt), gestione errore password, protezione delle risorse tramite filtro di autenticazione	Superato
IT-REG	RegistrazioneController	Validazione Codice Identificativo Speciale, unicità credenziali, rollback transazionale in caso di errore	Superato
IT-PREN	PrenotazioneController	Rilevamento conflitti di prenotazione e reindirizzamento con errore senza persistenza dei dati	Superato
IT-EVE	EventiController	Ciclo di vita dell'evento: creazione (Supervisore), iscrizione, disiscrizione, gestione posti esauriti	Superato
IT-TICK	TicketController / SupervisoreTicketController	Aggiornamento stato ticket e inserimento automatico nello storico degli aggiornamenti	Superato
IT-RIS	RisorsaController / SupervisoreRisorseController	Approvazione e rifiuto richieste di risorsa, gestione date non valide	Superato
IT-TAS	SupervisoreTasseController / SpeseController	Creazione tassa e avvio corretto del flusso di pagamento	Superato

5. System Testing

Il System Testing ha l'obiettivo di validare il comportamento complessivo del sistema OikoNaos dal punto di vista dell'utente finale, simulando l'interazione reale tramite browser web.

Questa fase di collaudo verifica i principali flussi end-to-end dell'applicazione, includendo l'interfaccia grafica basata su JSP, la gestione delle sessioni applicative e la corretta integrazione tra i livelli di presentazione, logica di business e persistenza dei dati.

I test di sistema consentono di valutare il prototipo come un'unità completa, assicurando che le funzionalità critiche definite nel Test Plan risultino accessibili, coerenti e utilizzabili nelle condizioni operative previste.

Strategia adottata

I test di sistema sono stati implementati mediante **Selenium WebDriver**, utilizzando **Google Chrome** come browser di riferimento. L'automazione interagisce direttamente con l'interfaccia utente, simulando azioni reali quali:

- compilazione dei form;
- selezione di elementi HTML;
- click su pulsanti e link;
- navigazione tra le pagine;
- verifica dinamica degli elementi del DOM.

In presenza di componenti HTML5 complessi o flussi non completamente deterministici, sono state adottate **tecniche avanzate**, quali l'iniezione di codice JavaScript e la lettura dinamica degli attributi del DOM, al fine di garantire la riproducibilità dei test e la corretta validazione del comportamento lato client e lato server.

Gestione Prenotazioni – PrenotazioniSystemTest

Lo scenario di test prevede che un Coinquilino effettui la prenotazione di uno spazio comune attraverso l'interfaccia web.

Per gestire correttamente campi di selezione della data basati su input HTML5, il test utilizza l'iniezione di codice JavaScript, impostando in modo controllato le date di inizio e fine prenotazione. Il caso di test verifica inoltre la gestione dei conflitti di prenotazione: il sistema tenta di inserire due prenotazioni identiche sullo stesso intervallo temporale. L'esito atteso, e correttamente verificato, è il blocco della seconda richiesta con la visualizzazione di un messaggio di errore sull'interfaccia utente, confermando il rispetto delle regole di esclusività temporale applicate dal backend.

Gestione Eventi – EventiSystemTest

Il test copre l'intero ciclo di vita di un evento, coinvolgendo ruoli differenti:

- creazione dell'evento da parte del Supervisore;
- iscrizione di un Coinquilino;
- successiva disiscrizione;
- cancellazione dell'evento da parte del Supervisore.

Durante l'esecuzione del test viene verificata in modo dinamico la visibilità e disponibilità dei pulsanti "Iscriviti" e "Disiscriviti", in funzione:

- dello stato dell'utente;
- del numero di posti disponibili;
- dello stato temporale dell'evento (futuro, in corso o concluso).

Questo approccio consente di validare non solo la correttezza funzionale, ma anche la coerenza dell'interfaccia rispetto allo stato applicativo.

Richiesta Risorse Condivise – RisorseSystemTest

Il test relativo alle risorse condivise si concentra sulla corretta integrazione tra calendario dinamico lato client e regole di disponibilità lato server.

Attraverso la lettura dell'attributo data-occupate presente nel DOM, il test individua le date già riservate. Successivamente, viene calcolata in Java la prima data disponibile, che viene inserita automaticamente nel form mediante JavaScript.

Il test verifica che il calendario reagisca correttamente alle restrizioni imposte dal backend, garantendo la sincronizzazione tra interfaccia utente e logica applicativa.

Recupero Password – RecuperoPasswordSystemTest

Lo scenario simula un utente che ha dimenticato la propria password. Per evitare dipendenze da servizi esterni, il test utilizza un parametro nascosto (testMode) che consente di bypassare l'invio reale dell'e-mail di recupero.

Il flusso testato include:

- generazione del token di reset;
- validazione del token;
- impostazione della nuova password.

L'esito positivo del test conferma la corretta implementazione del meccanismo di recupero credenziali in un contesto completamente automatizzato.

Gestione Ticket – TicketSystemTest

I test di sistema relativi ai ticket hanno l'obiettivo di validare il corretto funzionamento del **modulo di segnalazione e gestione delle richieste di supporto**.

Lo scenario principale prevede che un Coinquilino autenticato possa creare un nuovo ticket, compilando il form con titolo, descrizione, categoria e priorità. Il test automatizzato simula l'interazione completa con l'interfaccia JSP, verificando che i dati inseriti vengano correttamente acquisiti dal sistema e persistiti nel database. Vengono inoltre verificati i comportamenti successivi alla creazione, quali:

- la corretta visualizzazione del ticket nella lista personale dell'utente;
- l'assegnazione automatica dello stato iniziale del ticket;
- la visibilità delle informazioni in base al ruolo dell'utente.

L'esito positivo del test conferma l'integrazione corretta tra interfaccia, logica applicativa e persistenza dei dati anche per il modulo di gestione dei ticket.

Sicurezza Autenticazione – AuthSystemTest

Questo gruppo di test verifica i meccanismi di controllo accessi e protezione delle risorse riservate. Viene simulato il tentativo di accesso diretto a pagine protette (ad esempio /supervisore/home.jsp) da parte di utenti non autenticati o privi dei permessi necessari.

Il sistema intercetta correttamente tali richieste, reindirizzando l'utente verso la pagina di login o verso una home consentita, confermando il corretto funzionamento dei filtri di autenticazione e autorizzazione.

I test di sistema confermano che i principali flussi operativi del prototipo *OikoNaos* risultano correttamente funzionanti dal punto di vista dell'utente finale. L'interfaccia grafica, seppur minimale, risponde in modo coerente alle azioni dell'utente e riflette correttamente lo stato interno del sistema. Nel complesso, il System Testing fornisce una valida verifica end-to-end delle funzionalità critiche previste dal Test Plan, dimostrando l'integrazione efficace tra interfaccia, logica applicativa e persistenza dei dati.

6. Riepilogo Copertura

L'architettura di testing adottata, articolata su **tre livelli complementari**, consente di validare il sistema OikoNaos in modo progressivo e coerente:

1. **Unit Test (DAO):** Verificano l'integrità dei dati e la correttezza delle operazioni di accesso e persistenza, assicurando il corretto funzionamento delle query e delle transazioni sul database.
2. **Integration Test (Controller):** Validano la logica applicativa e il flusso di gestione delle richieste, inclusi i meccanismi di validazione, la gestione degli errori e l'interazione tra componenti applicativi e livello di persistenza.
3. **System Test (Interfaccia Utente):** Confermano la correttezza dei flussi end-to-end dal punto di vista dell'utente finale, garantendo un'esperienza d'uso fluida e priva di blocchi funzionali nelle principali funzionalità del sistema.

Questa strategia multilivello permette di individuare e isolare efficacemente eventuali anomalie, riducendo il rischio che errori non intercettati in fasi precedenti emergano in ambiente operativo.

Matrice di Copertura

- **Logica core (Prenotazioni, Risorse condivise):** Coperta in modo completo attraverso test di integrazione e di sistema, con verifica sia delle regole di business sia del comportamento dell'interfaccia utente.
- **Logica finanziaria (Tasse, Pagamenti):** I flussi principali in condizioni operative standard sono stati verificati, concentrandosi sui percorsi positivi e sull'integrazione con i moduli di supporto.
- **Sicurezza (Autenticazione, Registrazione e Controllo Accessi):** I meccanismi di autenticazione e autorizzazione sono stati verificati mediante test di sistema, validando il corretto funzionamento dei filtri di accesso alle risorse protette e del controllo degli accessi basato sui ruoli (RBAC). Gli aspetti di sicurezza applicativa, quali la prevenzione di SQL Injection e Cross-Site Scripting (XSS), risultano mitigati a livello implementativo tramite l'uso di prepared statement e tecnologie JSP standard, ma non sono stati oggetto di test automatici specifici.